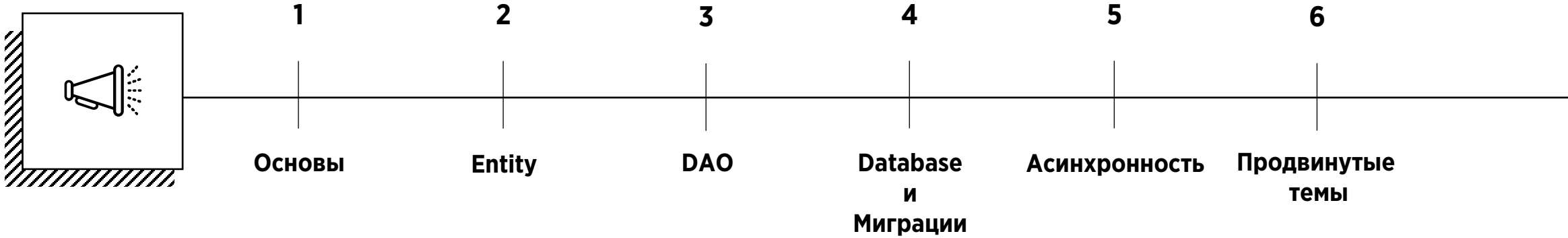
Локальная база данных

Содержание лекции



Основы: Зачем нужен Room и из чего он состоит.

Зачем нужен Room, если есть SQLite?



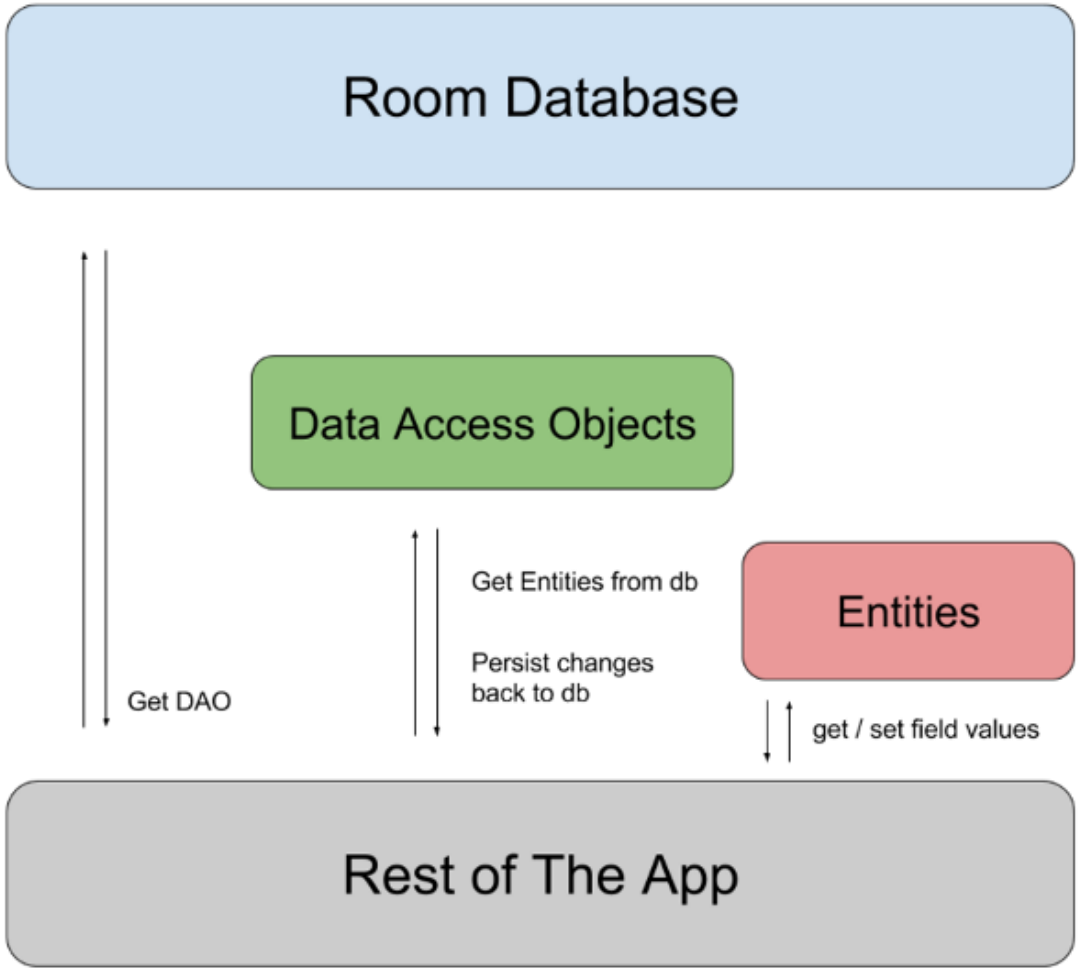
Room — это не новая база данных, а **умная оболочка (абстракция)** над SQLite, рекомендованная Google.

Он решает ключевые проблемы прямого использования SQLite:

- **Безопасность типов:** Проверяет ваши SQL-запросы на этапе компиляции. Если вы ошибетесь в имени таблицы или столбца, вы узнаете об этом сразу, а не когда приложение упадет у пользователя.
- **Меньше шаблонного кода:** Автоматически преобразует результаты запросов в ваши Kotlin/Java объекты. Больше не нужно вручную работать с Cursor.
- **Упрощение сложных задач:** Встроенная поддержка миграций, асинхронных запросов с помощью Flow/LiveData и удобное тестирование.

Три кита библиотеки Room

- @Entity (Сущность): Класс данных, описывающий таблицу в БД.
  - *Аналогия: чертеж таблицы.*
- @Dao (Data Access Object): Интерфейс с методами для доступа к данным (INSERT, SELECT, UPDATE...).
  - *Аналогия: пульт управления базой данных.*
- @Database (База данных): Абстрактный класс, который связывает Entity и DAO вместе. Точка входа для работы с БД.
  - *Аналогия: сам механизм базы данных.*



@Entity: Как правильно описывать таблицы данных.

Что такое @Entity?

@Entity — это аннотация для data class, которая превращает его в таблицу базы данных.

- Каждый **экземпляр** класса становится **строкой** в таблице.
- Каждое **поле** класса становится **столбцом** в таблице.

Это позволяет вам определять схему БД декларативно, без написания CREATE TABLE SQL-запросов.

Что такое @Entity?

```
@Entity
data class User(
    @PrimaryKey val id: Int, // Каждой таблице нужен первичный ключ
    val firstName: String?,
    val lastName: String?
)
```

Первичный ключ: @PrimaryKey

@PrimaryKey — обязательная аннотация, которая помечает одно или несколько полей как уникальный идентификатор строки.
Room использует этот ключ для эффективного поиска, обновления и удаления записей.

Автоматическая генерация ID:

- Если вы хотите, чтобы SQLite сам назначал уникальные ID (например, 1, 2, 3...), используйте autoGenerate = true.
- В этом случае поле должно быть типа Int или Long и быть изменяемым (var) со значением по умолчанию 0.

Первичный ключ: @PrimaryKey

```
@Entity
data class Product(
    @PrimaryKey(autoGenerate = true)
    var id: Long = 0, // Room сам подставит ID при вставке
    val name: String,
    val price: Double
)
```

Составной первичный ключ

Иногда уникальность строки определяется комбинацией нескольких полей.
В этом случае @PrimaryKey не используется, а ключи объявляются на уровне аннотации @Entity в свойстве primaryKeys.

Составной первичный ключ

```
@Entity(primaryKeys = ["userId", "magazineId"])
data class Subscription(
    val userId: Long,
    val magazineId: Long,
    val subscriptionDate: Long
)
```

В этой таблице может быть много записей с одним и тем же userId и много с одним и тем же magazineId, но комбинация (userId, magazineId) всегда будет уникальной.

Настройка имен: tableName и name

По умолчанию Room использует имена классов и полей "как есть" (например, класс User -> таблица User).

Хорошей практикой является использование snake_case для имен в SQL. Это можно настроить.

- @Entity(tableName = "...") — задает имя для таблицы.
- @ColumnInfo(name = "...") — задает имя для столбца.

Настройка имен: tableName и name

```
// Класс называется UserProfile, поле - creationTimestamp
@Entity(tableName = "user_profiles")
data class UserProfile(
    @PrimaryKey val id: Long,

    @ColumnInfo(name = "creation_timestamp")
    val creationTimestamp: Long
)
```

Игнорирование полей: @Ignore

Иногда в классе есть поля, которые **не нужно сохранять** в базу данных (например, временные данные для UI).
Для этого используется аннотация **@Ignore**. Room просто "не увидит" это поле при создании таблицы и работе с ней

Альтернатива для унаследованных полей: @Entity(ignoredColumns = ["..."])

Игнорирование полей: @Ignore

```
@Entity(tableName = "articles")
data class Article(
    @PrimaryKey val id: String,
    val content: String,

    // Это поле нужно только для UI, чтобы отметить выделенную статью.
    // В БД его не будет.
    @Ignore
    var isSelected: Boolean = false
)
```

Индексы для ускорения: @Index

Проблема: Если у вас большая таблица (тысячи строк) и вы часто ищете по одному и тому же полю (не по @PrimaryKey), запросы могут быть медленными.

Решение: Создать индекс для этого столбца. Индекс — это специальная структура данных, которая позволяет SQLite очень быстро находить строки по значению в индексированном столбце.

Индексы создаются через свойство indices в аннотации @Entity.

Индексы для ускорения: @Index

```
@Entity(indices = [Index(value = ["publish_date"])])
data class Article(
    @PrimaryKey val id: String,
    val title: String,
    @ColumnInfo(name = "publish_date")
    val publishDate: Long
)
// Теперь запросы вида "WHERE publish_date > ..." будут работать
быстрее.
```

Обеспечение уникальности

Иногда нужно гарантировать, что значение в столбце (или комбинации столбцов) не будет повторяться во всей таблице.
Это делается с помощью свойства unique = true внутри аннотации @Index.
При попытке вставить дубликат Room выбросит SQLiteConstraintException.

Пример: у каждого пользователя должен быть уникальный email

Обеспечение уникальности

```
@Entity(indices = [Index(value = ["email"], unique = true)])
data class User(
    @PrimaryKey(autoGenerate = true) var id: Long = 0,
    val email: String,
    val name: String
)
```

Встраивание объектов: @Embedded

Проблема: У вас есть класс, который логически является частью другого, например, Address внутри User. Вы не хотите создавать для адреса отдельную таблицу.

Решение: Использовать аннотацию @Embedded. Room «развернет»" все поля вложенного класса прямо в таблицу родительского класса.

Встраивание объектов: @Embedded

```
data class Address(
    val street: String?,
    val city: String?
)

@Entity
data class User(
    @PrimaryKey val id: Int,
    val name: String,
    @Embedded
    val address: Address?
)
// В результате таблица 'User' будет иметь столбцы:
// id, name, street, city
```

@Dao:
Как создавать запросы для доступа к данным.

Что такое @Dao?

DAO (Data Access Object) — это интерфейс, который вы определяете для взаимодействия с базой данных.
Вы объявляете методы, а Room автоматически генерирует их реализацию во время компиляции.

Зачем это нужно?

- Разделение ответственности: Ваш код (ViewModel, Repository) работает с простым интерфейсом, а не с деталями SQL.
- Безопасность типов: Room проверяет все ваши SQL-запросы в аннотациях @Query.
- Легкость тестирования: Вы можете легко создать "фейковую" реализацию DAO для юнит-тестов.

Что такое @Dao?

```
@Dao
interface UserDao {
    // Здесь будут методы для работы с пользователями
}
```

Простая операция: @Insert

Аннотация @Insert помечает метод для вставки данных в таблицу.
Метод может принимать один объект или список объектов.

Стратегия поведения при конфликте (onConflict):

- OnConflictStrategy.ABORT (по умолчанию): Если запись с таким @PrimaryKey уже есть, операция прерывается с ошибкой.

- OnConflictStrategy.REPLACE: Если запись существует, она будет перезаписана новой. Очень полезно для кэширования данных.

- OnConflictStrategy.IGNORE: Если запись существует, новая просто игнорируется.

Простая операция: @Insert

```
@Dao
interface UserDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    fun insertOrUpdate(user: User)

    @Insert
    fun insertAll(users: List<User>)
}
```

Что такое @Dao?

DAO (Data Access Object) — это интерфейс, который вы определяете для взаимодействия с базой данных.
Вы объявляете методы, а Room автоматически генерирует их реализацию во время компиляции.

Зачем это нужно?

- Разделение ответственности: Ваш код (ViewModel, Repository) работает с простым интерфейсом, а не с деталями SQL.
- Безопасность типов: Room проверяет все ваши SQL-запросы в аннотациях @Query.
- Легкость тестирования: Вы можете легко создать "фейковую" реализацию DAO для юнит-тестов.

Что такое @Dao?

```
@Dao
interface UserDao {
    // Здесь будут методы для работы с пользователями
}
```

Простые операции: @Update и @Delete

@Update и @Delete работают очень похоже: они находят запись в таблице по её первичному ключу (@PrimaryKey) и выполняют соответствующее действие.
Если запись с таким ключом не найдена, ничего не происходит.
Они также могут принимать один объект или список.
Опционально могут возвращать int — количество измененных или удаленных строк.

Простые операции: @Update и @Delete

```
@Dao
interface UserDao {
    @Update
    fun update(user: User): Int
    // Вернет 1, если пользователь был обновлен, иначе 0

    @Delete
    fun delete(user: User)
}
```

Запросы на чтение: @Query

Для всех операций, кроме простых вставок/обновлений/удалений, используется аннотация @Query.
Вы пишете внутри нее стандартный SQL-запрос.

Главная фича Room: этот SQL-запрос проверяется на этапе компиляции. Room «знает» вашу схему данных и не даст вам скомпилировать код с ошибкой в имени таблицы или столбца.

Запросы на чтение: @Query

```
@Dao
interface UserDao {
    @Query("SELECT * FROM user")
    // Room знает, что таблица называется 'user'
    fun getAll(): List<User>
}
```

Передача параметров в @Query

Чтобы делать запросы динамическими, вы можете передавать в них параметры из методов вашего DAO.
В SQL-строке параметр помечается двоеточием (:), за которым следует имя аргумента из метода.

Передача параметров в @Query

```
@Dao
interface UserDao {
    // Найти пользователя по ID
    @Query("SELECT * FROM user WHERE id = :userId")
    fun findById(userId: Long): User?

    // Найти пользователей по имени и минимальному возрасту
    @Query("SELECT * FROM user WHERE first_name LIKE :name AND age >
:minAge")
    fun findByNameAndAge(name: String, minAge: Int): List<User>
}
```

Возвращение части данных (Custom POJO)

Проблема: Часто для экрана нужно не все 20 полей объекта User, а только имя и фамилия. Запрашивать лишние данные неэффективно.

Решение:

1. Создать простой data class только с нужными полями (без аннотации @Entity).

2. Написать @Query, который выбирает конкретные столбцы и возвращает экземпляр этого класса.

Возвращение части данных (Custom POJO)

```
// 1. Класс для результата
data class UserFullName(
    @ColumnInfo(name = "first_name") val firstName: String?,
    @ColumnInfo(name = "last_name") val lastName: String?
)

// 2. DAO-метод
@Dao
interface UserDao {
    @Query("SELECT first_name, last_name FROM user WHERE id = :userId")
    fun getUserFullName(userId: Long): UserFullName
}
```

Запросы к нескольким таблицам: JOIN

Room полностью поддерживает стандартные SQL JOIN для объединения данных из нескольких таблиц.
Результат такого запроса обычно возвращается в виде кастомного POJO, как мы рассмотрели на предыдущем слайде.

Пример: получить имя пользователя и название книги, которую он взял

Запросы к нескольким таблицам: JOIN

```
// POJO для результата
data class LoanInfo(val userName: String, val bookTitle: String)

@Dao
interface LoanDao {
    @Query("""
        SELECT u.name as userName, b.title as bookTitle
        FROM user u
        JOIN book b ON u.id = b.user_id
        WHERE u.is_active = 1
    """)
    fun getActiveLoansInfo(): List<LoanInfo>
}
```

Атомарность запросов: @Transaction

Проблема: Некоторые операции требуют выполнения нескольких SQL-запросов (например, JOIN под капотом у @Relation или несколько вставок подряд). Что если один из них выполнится, а второй упадет с ошибкой? Данные останутся в несогласованном состоянии.

Решение: Аннотация @Transaction. Она гарантирует, что все запросы внутри метода будут выполнены как единая атомарная операция.

- Либо все выполнятся успешно.
- Либо, в случае ошибки, все будут отменены.

Правило: Всегда используйте @Transaction для методов DAO, которые выполняют несколько операций записи или сложные запросы с @Relation.

Атомарность запросов: @Transaction

```
@Dao
interface MyDao {
    @Transaction
    fun updateData() {
    // В теле этого метода можно вызывать другие методы DAO
        deleteAllUsers()
        insertNewUsers(users)
    }
}
```

Продвинутые параметры: передача коллекций

Room «понимает», когда вы передаете в метод коллекцию, и может использовать ее в SQL-запросе, например, для оператора IN.
Это позволяет выполнять запросы с переменным числом аргументов.

Пример: найти всех пользователей из определенного списка регионов

Продвинутые параметры: передача коллекций

```
@Dao
interface UserDao {
    @Query("SELECT * FROM user WHERE region IN (:regions)")
    fun findUsersFromRegions(regions: List<String>): List<User>
}

// Пример вызова:
val users = userDao.findUsersFromRegions(listOf("EU", "US"))
```

Продвинутые результаты: возврат Map (Multimap)

Начиная с Room 2.4, запросы могут возвращать Map, что очень удобно для представления отношений «один-ко-многим».
Room автоматически сгруппирует результаты запроса в карту.

Пример: получить карту, где ключ — Артист, а значение — список его Альбомов

Продвинутые результаты: возврат Map (Multimap)

```
// POJO для Артиста и Альбома
data class Artist(...)
data class Album(...)

@Dao
interface MusicDao {
    @Query("""
        SELECT * FROM Artist
        LEFT JOIN Album ON Artist.artistName = Album.artistCreatorName
    """)
    @Transaction // Транзакция обязательна для таких запросов
    fun getArtistsWithAlbums(): Map<Artist, List<Album>>
}
```

Полнотекстовый поиск: @Fts4

Проблема: Обычный LIKE '%...%' в SQL медленный и неэффективный для поиска по большим текстам (например, по содержанию статей).

Решение: SQLite имеет встроенный мощный механизм полнотекстового поиска (FTS - Full-Text Search). Room дает к нему простой доступ.

Для этого сущность помечается аннотацией @Fts3 или @Fts4.

Полнотекстовый поиск: @Fts4

```
@Fts4
@Entity(tableName = "articles")
data class Article(
    val title: String,
    val content: String
)

// В DAO можно использовать специальный синтаксис для поиска
@Query("SELECT * FROM articles WHERE articles MATCH :query")
fun searchArticles(query: String): List<Article>
```

@Database и Миграции: Как управлять базой данных и ее версиями.

Что такое @Database?

@Database — это главный класс, который «собирает» всю базу данных воедино. Он является точкой входа для доступа ко всем DAO.

Основные требования к классу:

1. Должен быть абстрактным и наследовать RoomDatabase.
2. В аннотации нужно перечислить все entities (классы-таблицы).
3. Указать version (версию схемы БД). Начинаем с 1.
4. Содержать абстрактный метод для каждого DAO.

Room сам сгенерирует конкретную реализацию этого класса.

Что такое @Database?

```
@Database(entities = [User::class, Product::class], version = 1)
abstract class AppDatabase : RoomDatabase() {

    abstract fun userDao(): UserDao
    abstract fun productDao(): ProductDao

}
```

Создание экземпляра БД

Экземпляр класса @Database не создается напрямую через конструктор. Для этого используется специальный билдер — Room.databaseBuilder.

Основные параметры билдера:

- context — контекст приложения.
- AppDatabase::class.java — класс вашей базы данных.
- "app_database.db" — имя файла, в котором будут храниться данные на устройстве.

Метод .build() запускает процесс создания.

Создание экземпляра БД

```
// Обычно этот код располагается в классе Application или в DI-модуле

val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java,
    "app_database.db"
).build()
```

Паттерн Singleton для БД

Правило: В приложении должен быть только один экземпляр класса AppDatabase.

Почему?

- Производительность: Создание экземпляра БД — дорогая операция.
- Согласованность данных: Несколько экземпляров, работающих с одним файлом, могут привести к ошибкам и повреждению данных.

Как реализовать?

- Через object в Kotlin.
- С помощью DI-фреймворка (Hilt, Koin), который сконфигурирован для создания синглтона.

Паттерн Singleton для БД

```
@Database(...)
abstract class AppDatabase : RoomDatabase() {
    // ... dao-методы ...

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getInstance(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(...).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Жизненный цикл: проблема изменений

Представьте, что ваше приложение уже в Google Play. У пользователей установлена version = 1.
В новой версии вы решили добавить в User поле age и изменили Entity:

Паттерн Singleton для БД

```
@Entity
data class User(
    @PrimaryKey val id: Int,
    val name: String,
    val age: Int // Новое поле!
)
```

Жизненный цикл: проблема изменений

Вы обязаны изменить версию базы данных:
@Database(..., version = 2)

Что произойдет у пользователя при обновлении?

- Room увидит, что версия на устройстве (1) не совпадает с ожидаемой версией (2).
- Он не знает, как обновить схему.
- Приложение упадет с IllegalStateException!

Решение: Миграции

Миграция — это объект, в котором вы описываете SQL-запросы для перехода с одной версии схемы на другую.

Как это работает:

- Создается объект Migration, указываются начальная и конечная версии.
- В методе migrate() пишется SQL-код для изменения схемы.
- Миграция регистрируется в билдере через .addMigrations().

Пример: миграция с версии 1 на 2 для добавления столбца age

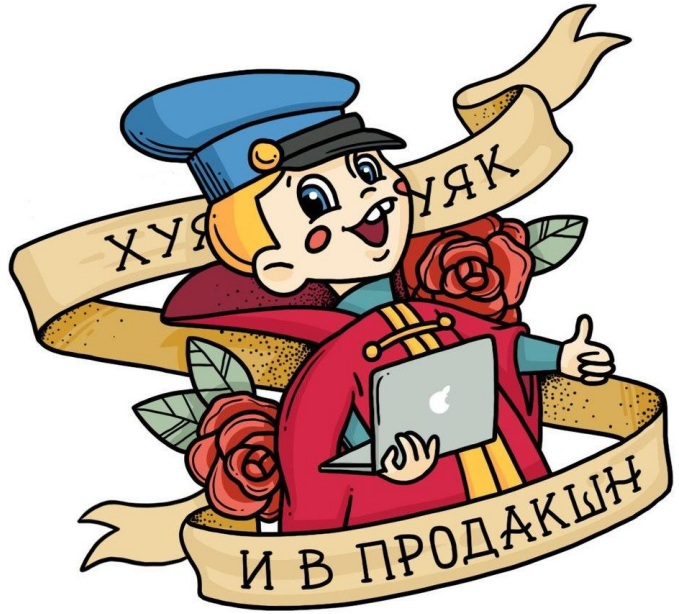
Паттерн Singleton для БД

```
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(database: SupportSQLiteDatabase) {
        database.execSQL("ALTER TABLE User ADD COLUMN age INTEGER NOT
NULL DEFAULT 0")
    }
}

// При создании БД:
val db = Room.databaseBuilder(...)
    .addMigrations(MIGRATION_1_2)
    .build()
```

Решение: Миграции

«Ленивый» вариант: .fallbackToDestructiveMigration() — если не хочется писать миграцию, можно просто удалить все данные пользователя. Использовать с большой осторожностью!



Асинхронность: Как работать с БД, не блокируя пользовательский интерфейс

Главное правило: БД вне UI потока

Правило №1 при работе с Room: Любой доступ к базе данных (чтение или запись) — это дисковая операция. Она может быть медленной.

Проблема: Если выполнять дисковую операцию в главном (UI) потоке, интерфейс приложения зависнет. Если зависание будет долгим, система покажет диалог «Приложение не отвечает» (ANR).

Решение от Room: Чтобы защитить вас от этой ошибки, Room запрещает вызывать методы DAO в главном потоке.

- При попытке это сделать, приложение упадет с ошибкой IllegalStateException: Cannot access database on the main thread..

Вывод: Все запросы к базе данных должны выполняться асинхронно в фоновом потоке.

Современный подход: Kotlin Coroutines

Room имеет первоклассную поддержку Kotlin Coroutines, что делает асинхронную работу очень простой. Для этого нужно подключить зависимость room-ktx.

Два основных сценария:

- Одноразовые операции (One-shot):
  - Запись, обновление, удаление, разовое чтение.
  - Решение: пометить метод в DAO ключевым словом suspend.

- Наблюдаемые запросы (Observable):
  - Получение данных, которые автоматически обновляются при изменении в таблице.
  - Решение: вернуть из метода DAO Flow<T>.

Современный подход: Kotlin Coroutines

```
@Dao
interface UserDao {
    // 1. Одноразовая операция
    @Insert
    suspend fun insert(user: User)

    // 2. Наблюдаемый запрос
    @Query("SELECT * FROM user")
    fun getAllUsersStream(): Flow<List<User>>
}
```

Пример: suspend и viewModelScope

suspend-функции DAO должны вызываться из корутины.
В Android-архитектуре для этого идеально подходит viewModelScope из ViewModel.
viewModelScope автоматически запускает корутину в нужном для работы с Room потоке (Dispatchers.IO) и отменяет ее, когда ViewModel уничтожается.

[Иконка XML-кнопки «Сохранить»] ->
[Fragment.setOnClickListener] ->
[ViewModel.addUser()] ->
[viewModelScope.launch] ->
[Фоновый поток выполняет userDao.insert()]

Пример: suspend и viewModelScope

```
class UserViewModel(private val userDao: UserDao) : ViewModel() {

    fun addUser(userName: String) {
        // Запускаем корутину, которая не блокирует UI
        viewModelScope.launch {
            val newUser = User(name = userName)
            userDao.insert(newUser) // Вызываем suspend-функцию
        }
    }
}
```

Пример: suspend и viewModelScope

```
// In MyFragment.kt
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    // ...
    saveButton.setOnClickListener {
        val name = nameEditText.text.toString()
        viewModel.addUser(name)
    }
}
```

Flow и подписка на обновления

Чтобы удобно использовать Flow в связке с XML, его обычно преобразуют в LiveData.

Шаг 1: Во ViewModel конвертируем Flow в LiveData.

- Используется extension-функция .asLiveData() из библиотеки lifecycle-livedata-ktx.

Шаг 2: Во Fragment подписываемся на LiveData.

- Метод .observe() будет автоматически получать обновления.

[Иконка БД] -> [Flow<...> в DAO] -> [.asLiveData() в ViewModel] -> [.observe() во Fragment] -> [Иконка RecyclerView со списком].

Пример: suspend и viewModelScope

```
class UserListViewModel(private val userDao: UserDao) : ViewModel() {
    // Преобразуем холодный Flow в горячий, кэширующий LiveData.
    val users: LiveData<List<User>> = userDao.getAllUsersStream().asLiveData()
}
```

Пример: suspend и viewModelScope

```
// In UserListFragment.kt
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    // ... настройка RecyclerView и Adapter ...
    val adapter = UserAdapter()
    recyclerView.adapter = adapter

    // Подписываемся на изменения. viewLifecycleOwner гарантирует,
    // что подписка будет активна только пока жив UI фрагмента.
    viewModel.users.observe(viewLifecycleOwner) { updatedUserList ->
        adapter.submitList(updatedUserList) // Обновляем данные в RecyclerView
    }
}
```

Продвинутые темы: Связи между таблицами, представления и другое.

Связи между таблицами: @Relation

Проблема: В реляционных базах данных сущности часто связаны друг с другом (у пользователя есть плейлисты, в плейлисте есть песни). Room запрещает прямые ссылки на другие @Entity внутри одного @Entity.

Решение: Вместо этого связи определяются на уровне запросов.

Два основных подхода:

- Ручные JOIN: Мы уже рассмотрели их. Гибко, но требует написания SQL.
- @Relation: Декларативный способ. Вы описываете связь с помощью аннотации, а Room сам выполняет нужные запросы под капотом.

@Relation позволяет автоматически подгружать связанные сущности.

Связь «Один-ко-многим»

Проблема: В реляционных базах данных сущности часто связаны друг с другом (у пользователя есть плейлисты, в плейлисте есть песни). Room запрещает прямые ссылки на другие @Entity внутри одного @Entity.

Решение: Вместо этого связи определяются на уровне запросов.

Два основных подхода:

- Ручные JOIN: Мы уже рассмотрели их. Гибко, но требует написания SQL.
- @Relation: Декларативный способ. Вы описываете связь с помощью аннотации, а Room сам выполняет нужные запросы под капотом.

@Relation позволяет автоматически подгружать связанные сущности.

Связь «Один-ко-многим»

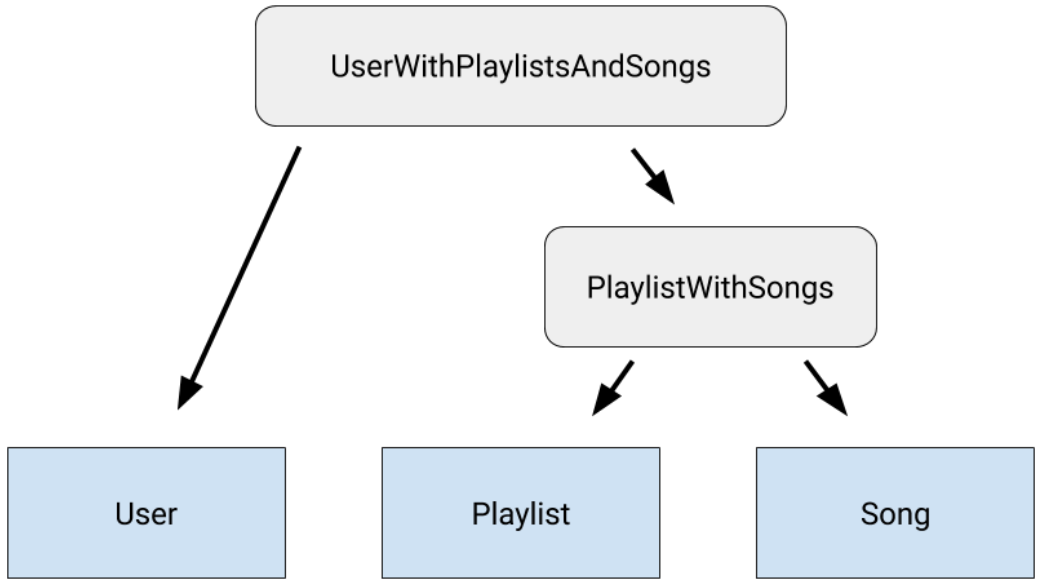
```
data class UserWithPlaylists(
    @Embedded val user: User,
    @Relation(
        parentColumn = "id", // Поле в родительской таблице (User)
        entityColumn = "userCreatorId" // Поле в дочерней таблице (Playlist)
    )
    val playlists: List<Playlist>
)

@Dao
interface UserDao {
    @Transaction // Обязательно для @Relation
    @Query("SELECT * FROM User")
    fun getUsersWithPlaylists(): List<UserWithPlaylists>
}
```

Связь «Многие-ко-многим»

Пример: Одна Песня (Song) может быть во многих Плейлистах (Playlist), и один Плейлист содержит много Песен.

Решение: Создается третья, связующая таблица (Junction Table), которая хранит пары (playlistId, songId).



Связь «Многие-ко-многим»

```
// 1. Три сущности
@Entity data class Playlist(...)
@Entity data class Song(...)
@Entity(primaryKeys = ["playlistId", "songId"])
data class PlaylistSongCrossRef(val playlistId: Long, val songId: Long)
```

Связь «Многие-ко-многим»

```
// 2. POJO для результата с @Relation и связующей таблицей
data class PlaylistWithSongs(
    @Embedded val playlist: Playlist,
    @Relation(
        parentColumn = "playlistId",
        entityColumn = "songId",
        associateBy = Junction(PlaylistSongCrossRef::class) // Указываем
связующую таблицу
    )
    val songs: List<Song>
)
```

Представления: @DatabaseView

Что это? Способ создать «виртуальную» read-only таблицу на основе сложного SELECT-запроса.

Зачем? Чтобы не дублировать один и тот же сложный JOIN-запрос в разных методах DAO.

Как использовать:

1. Создать класс, аннотированный @DatabaseView с вашим SQL-запросом.
2. Зарегистрировать его в классе @Database через свойство views.
3. Использовать в DAO так же, как и обычную @Entity, но только для чтения.

Представления: @DatabaseView

```
@DatabaseView("""
    SELECT user.name, COUNT(playlist.id) as playlistCount
    FROM user
    LEFT JOIN playlist ON user.id = playlist.userCreatorId
    GROUP BY user.id
""")
data class UserWithPlaylistCount(val name: String, val playlistCount: Int)
```

Предварительное заполнение БД

Что это? Возможность включить в ваше приложение (.apk) уже готовую базу данных с начальными данными.

Зачем? Для приложений, которым нужны данные с первого запуска (справочники, каталоги, начальные настройки).

Как использовать:

1. Поместите файл с базой данных SQLite в папку src/main/assets/.
2. При создании экземпляра @Database вызовите метод .createFromAsset().

Важно: Схема вашей БД в коде (@Entity) должна точно совпадать со схемой в файле.

Представления: @DatabaseView

```
val db = Room.databaseBuilder(
    context,
    AppDatabase::class.java, "app.db"
)
.createFromAsset("database/myapp.db") // Путь внутри папки assets
.build()
```